



## BÀI 07: CÁC THÀNH PHẦN SWING

---

- ❑ Tổng quan các thành phần SWING
  - ❑ JOptionPane
  - ❑ JRadioButton
  - ❑ JPanel
  - ❑ JCheckBox
  - ❑ JTable
  - ❑ TableModel
  - ❑ Tổng hợp kiến thức phần java AWT và SWING, JDBC áp dụng xây dựng ứng dụng kết nối cơ sở dữ liệu
- ❑ Bài tập



# TỔNG QUAN CÁC THÀNH PHẦN SWING

---

## ❑ Tổng quan các thành phần SWING

Swing được xây dựng dựa trên AWT:

- ✓ Swing sử dụng một số lớp cơ bản của AWT, đặc biệt là các lớp xử lý đồ họa và sự kiện. Ví dụ: các lớp như `java.awt.Component`, `java.awt.Container`, và `java.awt.LayoutManager` đều được Swing kế thừa.
- ✓ Swing mở rộng và thay thế các thành phần GUI của AWT để cung cấp thêm tính năng và khả năng tùy chỉnh.
- ✓ AWT phụ thuộc vào native components của hệ điều hành, có nghĩa là các thành phần giao diện của AWT được dựng dựa trên giao diện thực tế của hệ điều hành. Điều này đôi khi gây ra sự không nhất quán về giao diện trên các nền tảng khác nhau.
- ✓ Swing sử dụng thành phần lightweight (nhẹ), không phụ thuộc vào hệ điều hành. Thay vì sử dụng giao diện gốc, Swing tự vẽ giao diện của nó bằng Java, mang lại sự nhất quán trên mọi nền tảng.



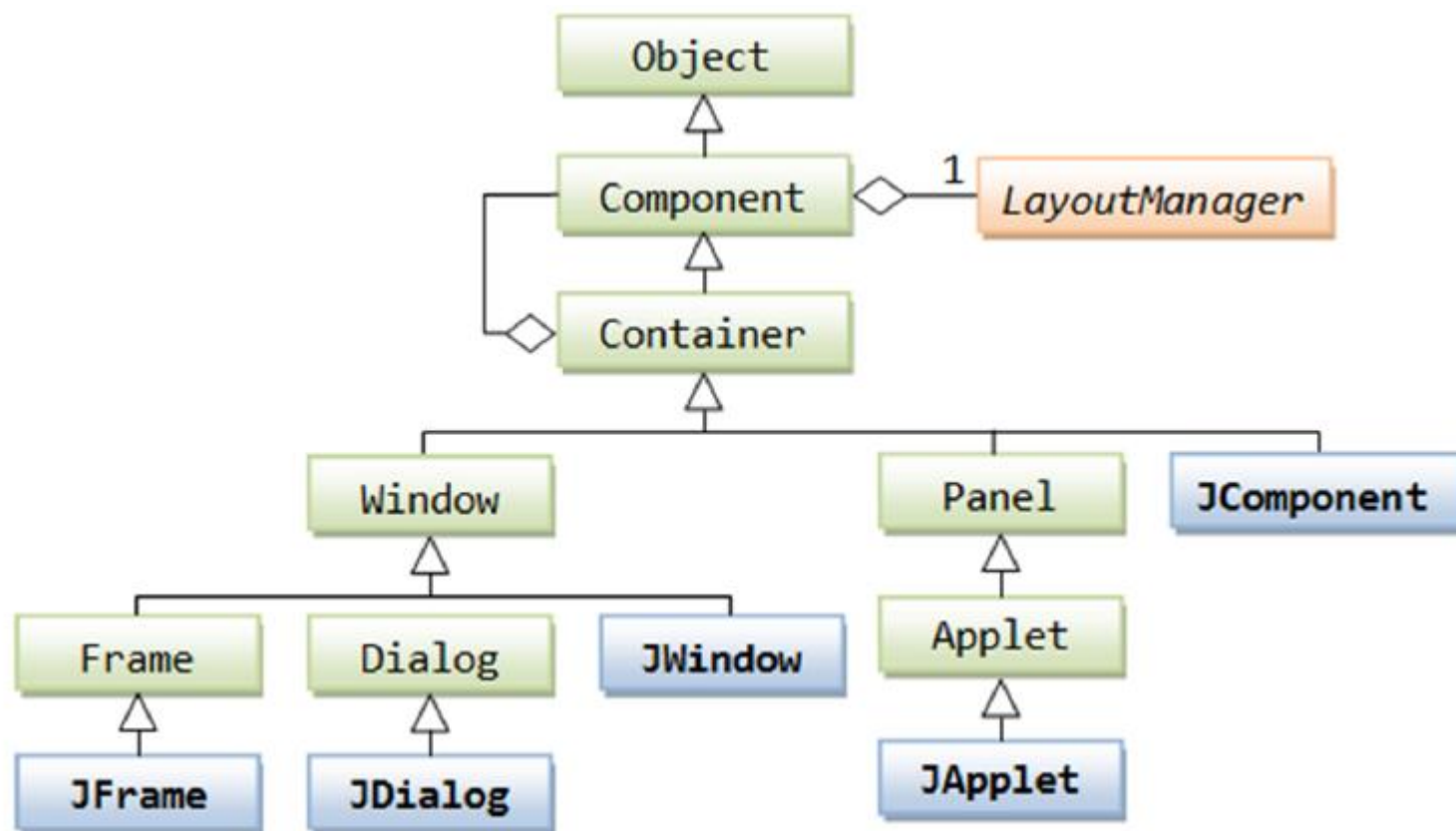
## TỔNG QUAN CÁC THÀNH PHẦN SWING

---

- ✓ Swing cung cấp một tập hợp phong phú các thành phần GUI, bao gồm các thành phần cơ bản, nâng cao, và thành phần điều hướng
- ✓ Swing tách biệt dữ liệu, giao diện, và logic xử lý giúp việc lập trình trở nên rõ ràng và dễ bảo trì. Mô hình MVC (Model-View-Controller)



# TỔNG QUAN CÁC THÀNH PHẦN SWING







# CÁC THÀNH PHẦN SWING

## ❑ JRadioButton, JPanel

```
JRadioButton yesButton      = new JRadioButton("Yes" , true);  
JRadioButton noButton       = new JRadioButton("No" , false);  
JRadioButton maybeButton = new JRadioButton("Maybe", false);
```

```
ButtonGroup bgroup = new ButtonGroup(); bgroup.add(yesButton);  
bgroup.add(noButton); bgroup.add(maybeButton);
```

```
JPanel radioPanel = new JPanel();  
radioPanel.setLayout(new GridLayout(3, 1));  
radioPanel.add(yesButton);  
radioPanel.add(noButton); radioPanel.add(maybeButton);  
radioPanel.setBorder(BorderFactory.createTitledBorder( BorderFactory.createEtchedBorder(), "Married?"));
```





# CÁC THÀNH PHẦN SWING

---

## ❑ JFileChooser

- ✓ Sử dụng JFileChooser dùng để lựa chọn một tập tin hoặc thư mục trong ổ đĩa.

```
JFileChooser fileChooser = new JFileChooser();  
int retval = fc.showOpenDialog(null);  
if (retval == JFileChooser.APPROVE_OPTION) {  
    File myFile = fc.getSelectedFile();  
    //Các thao tác xử lý file  
}
```

Cho phép chọn file/thư mục hoặc cả hai:

```
fc.setSelectionMode(JFileChooser.FILES_ONLY); //mặc định  
fc.setSelectionMode(JFileChooser.DIRECTORIES_ONLY);  
fc.setSelectionMode(JFileChooser.FILES_AND_DIRECTORIES);
```

- ❑ Tính năng chính của JTable dùng để hiển thị dữ liệu, dữ liệu được trình bày dưới dạng lưới (grid) gồm các hàng và cột.
- ❑ Cho phép chỉnh sửa các ô trong bảng.
- ❑ Kết hợp với JScrollPane để hiển thị dữ liệu lớn.
- ❑ Hỗ trợ thay đổi màu sắc, phong chữ, và bố cục.

The Header contains Column labels

| First Name | Last Name | Sport         | # of Years | Vegetarian                          |
|------------|-----------|---------------|------------|-------------------------------------|
| Kathy      | Smith     | Snowboarding  | 5          | <input type="checkbox"/>            |
| John       | Doe       | Rowing        | 3          | <input checked="" type="checkbox"/> |
| Sue        | Black     | Knitting      | 2          | <input type="checkbox"/>            |
| Jane       | White     | Speed reading | 20         | <input checked="" type="checkbox"/> |

Each Cell displays a data item

Each Column displays one type of data

## ❑ Các thành phần chính

- **Column (Cột):** Đại diện cho thuộc tính của dữ liệu.
- **Row (Hàng):** Đại diện cho một bản ghi hoặc dòng dữ liệu.
- **Cell (Ô):** Giao điểm của hàng và cột, chứa giá trị cụ thể.
- **Model:**
  - **TableModel:** Cấu trúc dữ liệu nguồn.
  - **DefaultTableModel:** Model mặc định cho JTable.

The Header contains Column labels

| First Name | Last Name | Sport         | # of Years | Vegetarian                          |
|------------|-----------|---------------|------------|-------------------------------------|
| Kathy      | Smith     | Snowboarding  | 5          | <input type="checkbox"/>            |
| John       | Doe       | Rowing        | 3          | <input checked="" type="checkbox"/> |
| Sue        | Black     | Knitting      | 2          | <input type="checkbox"/>            |
| Jane       | White     | Speed reading | 20         | <input checked="" type="checkbox"/> |
| ...        | ...       | ...           | ...        | ...                                 |

Each Cell displays a data item

Each Column displays one type of data



# JTable

```
public class TableDemo0 extends JFrame{
    protected JTable table;
}
    public TableDemo0(){
        Container pane = this.getContentPane();
        pane.setLayout(new BorderLayout());
        Object rowsData[][]={{"11","12","13"}, {"21","22","23"}, {"31","32","33"},
                               {"31","32","33"}, {"31","32","33"}, {"31","32","33"}, {"31","32","33"};
        Object colName[]={"Cot 1", "cot 2", "cot 3"};
        //rowsData[0]=colName;
        MyTableModel tv = new MyTableModel();
        table = new JTable(tv);
        table.getColumnModel().getColumn(0).setPreferredWidth(10);
        ///table=new JTable(rowsData,colName);
        JScrollPane jsp = new JScrollPane(table);
        pane.add(jsp, BorderLayout.CENTER);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setSize(400, 200);
        this.setVisible(true);
    }
}
    public static void main(String [] args){
        new TableDemo0();
    }
}
```



# TableModel

---

- ❑ Mục đích của TableModel là cung cấp dữ liệu hiển thị cho JTable
- ❑ Ngoài ra nó cung cấp cho JTable một vài thông tin khác như:
  - Số lượng dòng và số lượng cột trong bảng
  - Kiểu dữ liệu trong từng cột
  - Tiêu đề cho từng cột
  - Có cho phép sửa giá trị trong một ô hay không



# TableModel

| TableModel                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>◆getRowCount() : int</li><li>◆getColumnCount() : int</li><li>◆getValueAt(row : int, column : int) : Object</li><li>◆getColumnName(column : int) : String</li><li>◆getColumnClass(column : int) : Class</li><li>◆isCellEditable(row : int, column : int) : boolean</li><li>◆addTableModelListener(listener : TableModelListener) : void</li><li>◆removeTableModelListener(listener : TableModelListener) : void</li></ul> |



# AbstractTableModel

```
public class MyTableModel extends AbstractTableModel {
    public Object[][] values = {
        {1,"nguyen van A","Thai Nguyen",1980},{2," nguyen van B","Thai Nguyen",1981},
        {3,"nguyen thi C","Thai Binh",1981},{4," Tr n V C","Nam Dinh ",1981},
        {5," Tr n V Z","Nam Dinh ",1981}
    };
    public String[] conlumnName={"stt","Ho Ten","Dia Chi","Nam Sinh"};
    public int getRowCount() {
        System.out.println("so dong la: "+values.length);
        return values.length;
    }
    public int getColumnCount() {
        System.out.println("so cot la: "+values[0].length);
        return values[0].length;
    }
    public Object getValueAt(int rowIndex, int columnIndex) {
        System.out.println("goi getValueAt: "+rowIndex+" -"+columnIndex+
            "th  return: "+values[rowIndex][columnIndex]);
        return values[rowIndex][columnIndex];
    }
    public String getColumnName(int index){
        return conlumnName[index];
    }
}
```



# JTable

```
public class TableDemo0 extends JFrame{
    protected JTable table;
}
    public TableDemo0(){
        Container pane = this.getContentPane();
        pane.setLayout(new BorderLayout());
        Object rowsData[][]={{"11","12","13"}, {"21","22","23"}, {"31","32","33"},
                               {"31","32","33"}, {"31","32","33"}, {"31","32","33"}, {"31","32","33"};
        Object colName[]={"Cot 1", "cot 2", "cot 3"};
        //rowsData[0]=colName;
        MyTableModel tv = new MyTableModel();
        table = new JTable(tv);
        table.getColumnModel().getColumn(0).setPreferredWidth(10);
        ///table=new JTable(rowsData,colName);
        JScrollPane jsp = new JScrollPane(table);
        pane.add(jsp, BorderLayout.CENTER);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setSize(400, 200);
        this.setVisible(true);
    }
}
    public static void main(String [] args){
        new TableDemo0();
    }
}
```



# Bắt sự kiện cho JTable

```
public class SimpleTableTest1 extends JFrame implements ListSelectionListener{
    protected JTable table;
    public SimpleTableTest1() {
        Container pane = getContentPane();
        pane.setLayout(new BorderLayout());
        MyTableModel tv = new MyTableModel();
        table = new JTable(tv);
        table.getColumnModel().getColumn(0).setPreferredWidth(10);
        table.getColumnModel().getColumn(1).setPreferredWidth(70);
        table.getColumnModel().getColumn(2).setPreferredWidth(140);
        table.getColumnModel().getColumn(3).setPreferredWidth(20);
        table.getSelectionModel().addListSelectionListener(this);
        JScrollPane jsp = new JScrollPane(table);
        pane.add(jsp, BorderLayout.CENTER);
    }
    public void valueChanged(ListSelectionEvent e) {
        int row = table.getSelectedRow();
        int col = table.getSelectedColumn();
        if (row >= 0) {
            System.out.println("Chon dong " + row + " Cot " + col);
            System.out.println("get Value" + table.getValueAt(row, col));
        }
    }
}
```



# Xây dựng AbstractTableModel lấy dữ liệu từ DAO

```
public class MyTableModel extends AbstractTableModel{
    Vector tblData; // chứa dữ liệu bảng
    Vector colName; // chứa tiêu đề cột
    Vector colName1;
    public MyTableModel(ResultSet rs) throws SQLException{
        ResultSetMetaData rsmData=rs.getMetaData();
        int count=rsmData.getColumnCount();
        //int rowcount=rsmData.get
        colName=new Vector(count);
        colName1=new Vector(count);
        colName1.addElement("STT");
        colName1.addElement("Họ Tên");
        colName1.addElement("Địa Chỉ");
        colName1.addElement("Lớp");
        colName1.addElement("Năm Sinh");
        // lấy về tên của tiêu đề cột thông qua ResultSetMetaData
        for(int i=1;i<=count;i++)
            colName.addElement(rsmData洗getColumn洗Name(i));
        // duyệt ResultSet để get
```



# Xây dựng AbstractTableModel lấy dữ liệu từ DAO

```
tblData=new Vector();
while(rs.next()){
    // tạo ra 1 dòng dữ liệu và lấy dữ liệu từ rs
    Vector rowData=new Vector();
    for(int i=1;i<=count;i++){
        rowData.addElement(rs.getObject(i));
    }
    // thêm dòng dữ liệu vào tblData
    tblData.addElement(rowData);
}

public int getRowCount() {
    return tblData.size();
}

@Override
public int getColumnCount() {
    return colName.size();
}

@Override
public String getColumnName(int columnIndex) {
    return (String) colName.elementAt(columnIndex);
}

@Override
public Object getValueAt(int rowIndex, int columnIndex) {
    Vector rowData=(Vector)tblData.elementAt(rowIndex);
    return rowData.elementAt(columnIndex);
}
```



# Xây dựng AbstractTableModel lấy dữ liệu từ DAO

```
connecdata = new ConnectionDatabase();
    try {
        connecdata.connect();
        ResultSet rs = connecdata.getAllData();
        myModel = new MyTableModel(rs);
        table = new JTable(myModel);
        // thiết lập độ rộng cột cho Table
        table.getColumnModel().getColumn(0).setPreferredWidth(15);
        table.getColumnModel().getColumn(1).setPreferredWidth(130);
        table.getColumnModel().getColumn(2).setPreferredWidth(200);
        table.getColumnModel().getColumn(3).setPreferredWidth(50);
        table.getColumnModel().getColumn(4).setPreferredWidth(90);
        tblPane = new JScrollPane(table);
    } catch (SQLException|ClassNotFoundException e) {
        e.printStackTrace();
        JOptionPane.showMessageDialog(new JFrame(), "Không nạp được Driver"
            + e.toString(), "Lỗi kết nối", JOptionPane.ERROR_MESSAGE);
    }
```



# Xây dựng AbstractTableModel lấy dữ liệu từ DAO

Form kết nối CSDL

| ... | Họ Tên                | Địa Chỉ             | Lớp        | Năm Sinh   |
|-----|-----------------------|---------------------|------------|------------|
| 1   | Nguyen van ABC        | Thai Binh           | 11 A57     | 1981/02/02 |
| 2   | Tran Dai              | Ninh Binh           | K13B       | 2008/01/01 |
| 3   | Nguyen Lop Truong     | Vo Nhai 123         | K12A       | 1980/01/01 |
| 5   | Trinh Van Hong 987    | Hoang Van Thu       | 12A        | 2008       |
| 7   | Duong Mai Thuong 1... | Tan Lap Thai Nguyen | 12 ABC     | 2009       |
| ... | Boun Shou             | Vien Y              | Viet Ba... | 1990       |
| ... | Thi1235678            | Vien Chan Lao       | Viet Bac   | 2002       |
| ... | To Nguyen             | Thai Nguyen         | K12 A ...  | 1980       |
| ... | Nguyen Thi No 123     | Thai Binh           | K14A       | 1998       |

# Xây dựng AbstractTableModel lấy dữ liệu từ DAO

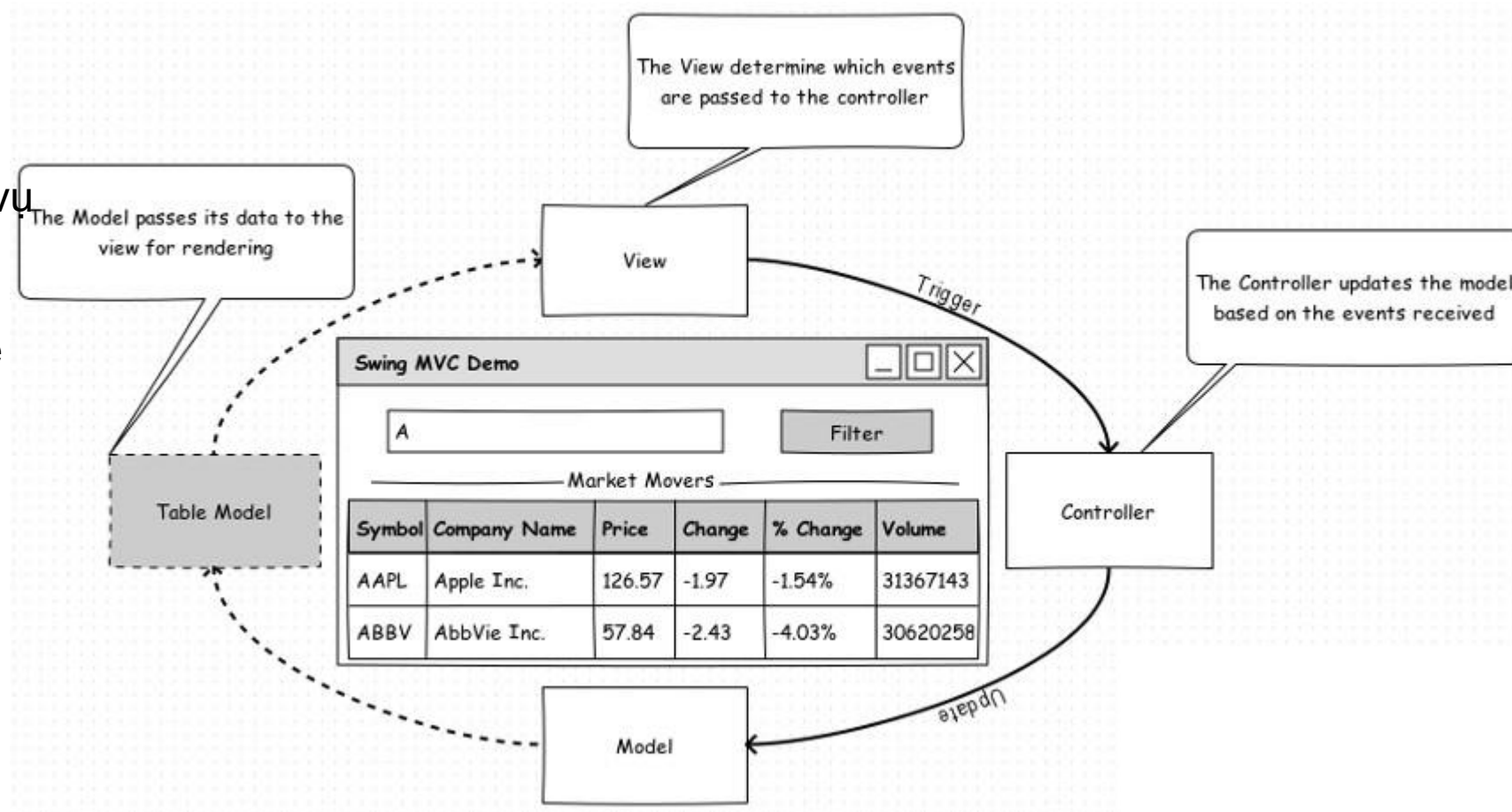
Mối quan hệ và vai trò của các thành phần trong mô hình MVC (Model-View-Controller) trong ứng dụng Java Swing.

## •Model

Là thành phần chịu trách nhiệm quản lý dữ liệu và logic nghiệp vụ.

•Model sẽ cung cấp dữ liệu cho View để hiển thị.

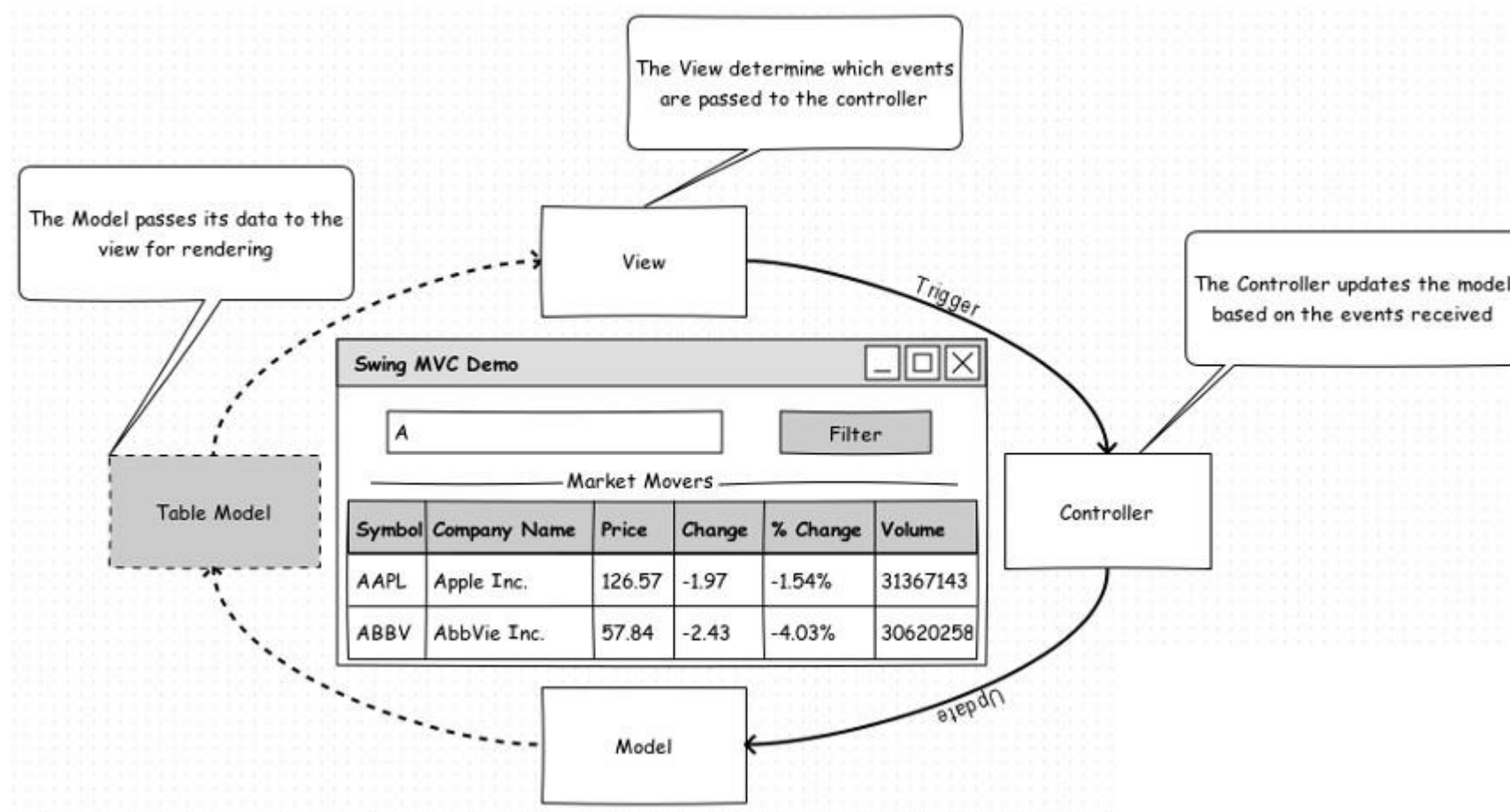
•**Table Model** được sử dụng để lưu trữ và quản lý dữ liệu của bảng.



# Xây dựng AbstractTableModel lấy dữ liệu từ DAO

## View:

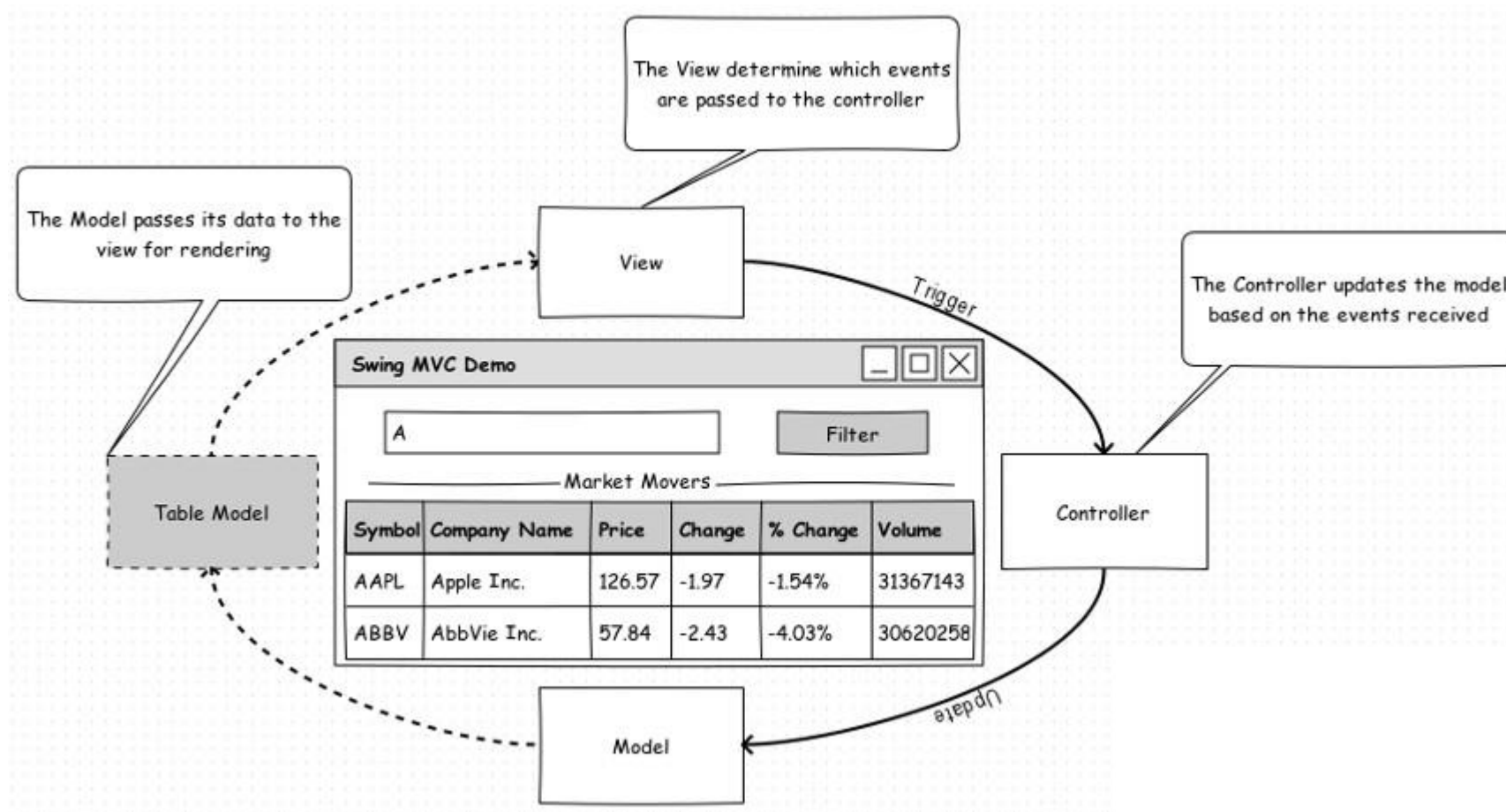
- ✓ Đảm nhận việc hiển thị giao diện người dùng. Xác định sự kiện nào cần gửi tới Controller.
- ✓ View được biểu diễn bởi giao diện "Swing MVC Demo" chứa các thành phần như ô nhập liệu, nút "Filter", và bảng dữ liệu.



# Xây dựng AbstractTableModel lấy dữ liệu từ DAO

## Controller:

- ✓ Xử lý các sự kiện từ người dùng và điều khiển luồng dữ liệu giữa Model và View.
- ✓ Controller nhận các sự kiện do View gửi đến (như nút "Filter" được nhấn).
- ✓ Controller sẽ cập nhật Model dựa trên các sự kiện nhận được.





# Xây dựng AbstractTableModel lấy dữ liệu từ DAO

---

❑ **Mối quan hệ giữa các thành phần:**

✓ **Luồng giao tiếp từ View đến Controller:**

Khi người dùng tương tác với View (như nhập dữ liệu hoặc nhấn nút), sự kiện được gửi đến Controller.

✓ **Controller cập nhật Model:**

Controller xử lý sự kiện và thực hiện các thay đổi trong Model (như thay đổi dữ liệu, lọc bảng, v.v.).

✓ **Model cập nhật View:**

Sau khi Model được cập nhật, nó sẽ thông báo cho View để hiển thị dữ liệu mới.



# BÀI THỰC HÀNH

---

- ❑ Bài tập thực hành

- ❑ Bài 1:

- ✓ Xây dựng ứng dụng quản lý sản phẩm với các chức năng:Hiển thị danh sách sản phẩm.Thêm sản phẩm mới.Cập nhật giá sản phẩm.

- ✓ **Yêu cầu:**

- Sử dụng Java Swing để thiết kế giao diện.
- Sử dụng JDBC để kết nối và thao tác với cơ sở dữ liệu.
- Áp dụng DAO và mô hình MVC.



# BÀI THỰC HÀNH

---

- ❑ Bài tập thực hành
- ❑ Bài 2:
  - ✓ Xây dựng ứng dụng quản lý danh sách nhân viên với các chức năng: Hiển thị danh sách nhân viên từ cơ sở dữ liệu, thêm một nhân viên mới vào cơ sở dữ liệu, xóa một nhân viên theo mã nhân viên.
  - ✓ Yêu cầu:
    - Sử dụng Java AWT để thiết kế giao diện.
    - Sử dụng JDBC để kết nối và thao tác với cơ sở dữ liệu.
    - Áp dụng DAO và mô hình MVC để tổ chức mã nguồn.



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG  
**KHOA CÔNG NGHỆ THÔNG TIN**

*Thank you!*

---

THÁI NGUYÊN, THÁNG .... NĂM ....